

Standard AES and HMAC-SHA256 Key Sizes

<https://gemini.google.com/app/04527d61a5fb2d9d>

User prompt: how many bytes is the standard for an aes key as well as hmacsha256 keys

Response: The key sizes for AES and HMAC-SHA256 depend on their respective cryptographic standards:

AES (Advanced Encryption Standard) AES supports three standard key sizes, defined by the number of bits:

- **16 bytes** (128 bits) – Used for AES-128.
- **24 bytes** (192 bits) – Used for AES-192.
- **32 bytes** (256 bits) – Used for AES-256 (the strongest standard).

HMAC-SHA256 While HMAC allows for variable-length keys, the standard recommendations are:

- **Recommended/Standard Size: 32 bytes** (256 bits). This matches the output size of the SHA-256 hash function.
- **Block Size Limit: 64 bytes** (512 bits). If you provide a key longer than 64 bytes, the HMAC algorithm will actually hash the key down to 32 bytes before using it.
- **Minimum:** While you can technically use a shorter key, using anything less than 32 bytes is strongly discouraged because it weakens the security of the hash.

In most modern systems, **32 bytes** is the go-to standard for both AES-256 and HMAC-SHA256.

Sources

1. [Advanced Encryption Standard - Wikipedia](#)
 2. [Key Size For SHA-256 HMAC : r/cryptography - Reddit](#)
-

User prompt: can you explain the RSA VerifyData method

Response: The `RSA.VerifyData` method is a cryptographic function used to confirm that a piece of data is authentic and hasn't been altered since it was signed. It validates a digital signature against the raw data using a sender's public key.

In the context of modern development frameworks like .NET (C#), `RSA.VerifyData` abstracts away the complex math of RSA and the manual hashing process, making signature verification a single method call.

How It Works Under the Hood

When you call `VerifyData`, the method performs three steps automatically:

1. **Hashes the Data:** It takes the raw, unencrypted data you provided and runs it through a specified hash algorithm (like SHA-256) to create a fixed-size digest.
2. **Decrypts the Signature:** It uses the provided RSA public key to "decrypt" the digital signature. Because the signature was created using the sender's private key, only their public key can unlock it. Unlocking it reveals the *original* hash the sender created.
3. **Compares the Hashes:** It compares the hash it just generated in Step 1 with the hash revealed in Step 2. If they match perfectly, the method returns `true`. If they differ by even a single bit, it returns `false`.

Standard Parameters

When invoking this method, you typically need to provide four components:

- **Data:** The raw byte array of the message or file you want to verify.

- **Signature:** The byte array containing the cryptographic signature provided by the sender.
- **Hash Algorithm:** The algorithm used to hash the data (e.g., `HashAlgorithmName.SHA256`). This must exactly match the algorithm the sender used when signing the data.
- **Padding Scheme:** The formatting applied to the signature to ensure security.
 - `RSASignaturePadding.Pkcs1` is the older, widely compatible standard.
 - `RSASignaturePadding.Pss` (Probabilistic Signature Scheme) is the newer, more mathematically secure standard recommended for modern applications.

Example Usage (C#)

Here is how `VerifyData` is typically implemented to check a signature:

```
using System;
using System.Security.Cryptography;
using System.Text;

public class RsaVerificationExample
{
    public static bool VerifyMessage(byte[] originalData, byte[] signature, string publicKeyXml)
    {
        using (RSA rsa = RSA.Create())
        {
            // Load the sender's public key
            rsa.FromXmlString(publicKeyXml);

            // Verify the data against the signature
            bool isAuthenticated = rsa.VerifyData(
                originalData,
                signature,
                HashAlgorithmName.SHA256,
                RSASignaturePadding.Pkcs1 // Must match the padding used during signing
            );

            return isAuthenticated;
        }
    }
}
```

If `VerifyData` returns `true`, it guarantees two things: **Integrity** (the data wasn't modified in transit) and **Non-repudiation** (only the holder of the corresponding private key could have created the signature).

User prompt: can you explain what an `InvalidSignatureException` is

Response: An `InvalidSignatureException` is an error thrown by an application, API, or cryptographic library when it cannot verify the mathematical authenticity of a digital signature.

At its core, this exception means the system evaluated the raw data, the provided signature, and the cryptographic key, and concluded that they do not match.

While raw methods like .NET's `RSA.VerifyData` simply return `false` when a signature fails, higher-level libraries (like JSON Web Token (JWT) validators, identity providers, or cloud platforms like AWS) will explicitly throw an `InvalidSignatureException` to halt the operation and prevent unauthorized access.

Here are the most common reasons this exception occurs:

- **Data Tampering or Corruption:** The payload was altered after it was originally signed. Even changing a single character or whitespace in the data will completely change the expected hash, causing the signature verification to fail.
- **Wrong Key Used:** The verifier is using the wrong public key or shared secret. For example, trying to verify a token signed by "Server A" using the public key of "Server B".
- **Algorithm or Padding Mismatch:** The sender signed the data using one algorithm (e.g., HMAC-SHA512 or RSA-PSS), but the receiver is trying to verify it using another (e.g., HMAC-SHA256 or RSA-PKCS1).
- **Encoding or Formatting Errors:** The signature string was corrupted during transmission. This often happens if a Base64-encoded signature loses trailing = padding characters, or if special characters like + or / are improperly URL-encoded.
- **Timestamp or Clock Skew (API Specific):** In cloud APIs (like AWS Signature Version 4), the signature is tied to a specific timestamp. If the clock on the machine sending the request is out of sync with the server by more than a few minutes, the generated signature is considered invalid.

When troubleshooting this error, the first step is almost always to log the exact byte arrays or strings of the payload, the key, and the signature immediately before the verification function is called to ensure nothing was subtly modified in transit.
